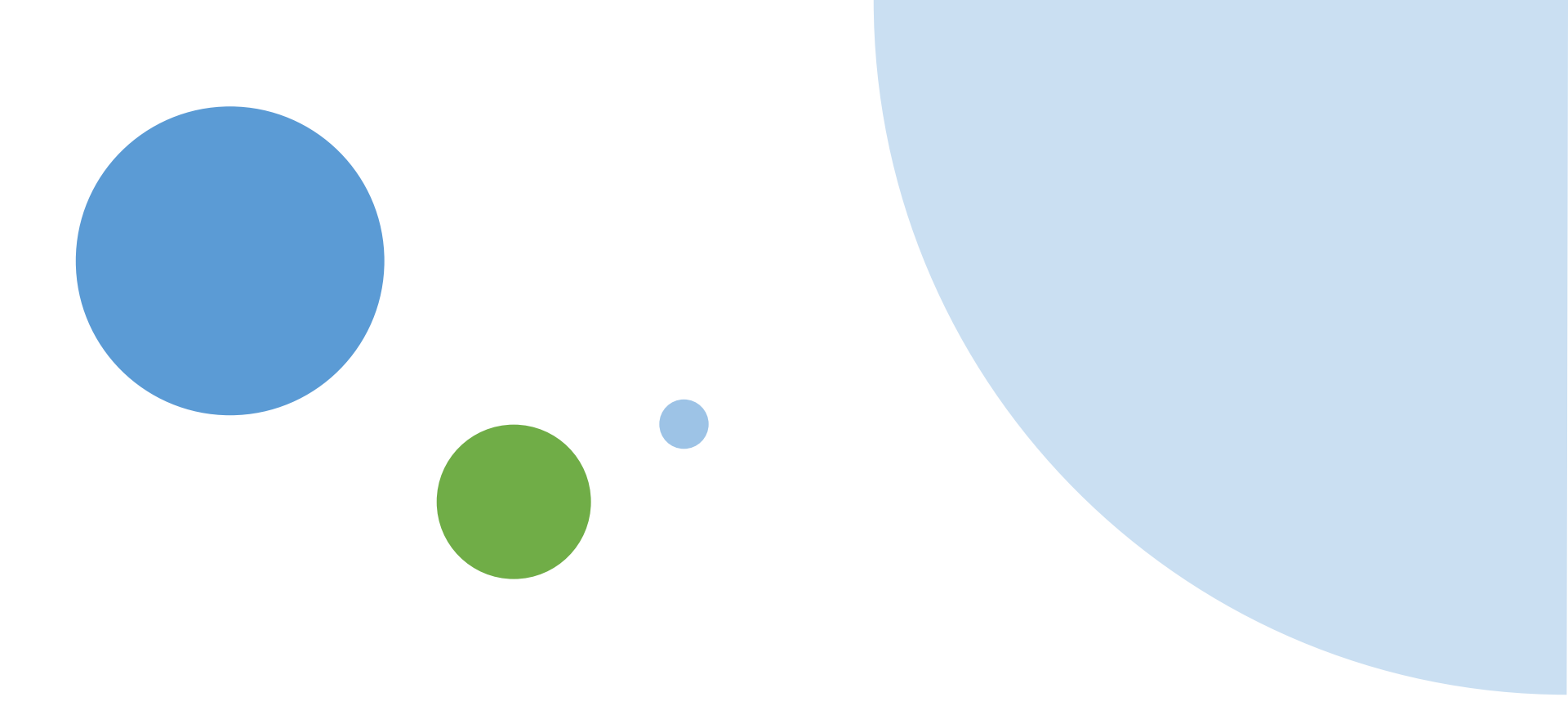




进阶算法思想

洛谷网校
cryozwq

单调栈

- 给定一个排列，求每个数最为最小值的最大区间
- 以这个数为基准，向左直到第一个小于它的为止（记为 l ），向右直到一个小于它的为止（记为 r ），那么 $[l+1, r-1]$ 这个区间就是以其为最小值的区间
- 这两个问题本质是一样的（只需要倒着做一遍即可）

单调栈

- 那么如何对于每个 a_i 找到其左边第一个比自己小的数呢？
- 核心思想：一个数的右边如果有一个数比自己小，那它之后就再也不会成为某个数左边第一个比自己小的数了（单调性）
- 我们从左到右进行处理，维护一个栈表示当前可能成为左边第一个比某个数小的数字
- 每次加入一个数，我们把栈里面所有大于这个数的都弹出，然后把这个数压入栈中
- 会发现完成弹出之后，栈顶的那个元素就是左边第一个比自己小的数，重复这个过程即可。

CF547B Mike and Feet

- 会发现由于我们遵循这个过程，整个栈中的元素一定是单调的（这也保证了我们每次弹出栈顶的正确性）

```
for(int i=1;i<=n;i++){  
    while(tp&&a[stk[tp]]>=a[i])tp--;  
    lst[i]=stk[tp];  
    stk[++tp]=i;  
}
```

- 这也就是一般所说的单调栈

CF547B Mike and Feet

定义一个序列的价值为这个序列的 $\min$ 。

给出长为 n 的序列，对每个 k ($1 \leq k \leq n$) 求出所有长为 k 的子区间价值的最大值。

$$1 \leq n \leq 2 \times 10^5, 1 \leq a_i \leq 10^9$$

CF547B Mike and Feet

- 转换一下思路，不去想对于每种长度的区间，有哪些不同的价值，而去思考对于每种价值，能够作为哪些长度区间的答案
- 如果有一个长度为 L 的区间价值为 v ，那么就会有长度为 1 到 $L - 1$ 的区间价值为 v （缩短这个区间即可）
- 因此对于每个 a_i ，我们找到其作为**最小值**的最大区间，那么长度小于等于其的长度都可以拿这个 a_i 当答案，全部算一遍取最大值即可（需要做一次后缀 max）
- 找这个区间的过程直接使用单调栈即可

CF547B Mike and Feet

- 参考实现

```
for(int i=1;i<=n;i++){
    int len=nxt[i]-lst[i]-1;
    ans[len]=max(ans[len],a[i]);
}
for(int i=n;i>=1;i--)ans[i]=max(ans[i],ans[i+1]);
for(int i=1;i<=n;i++)cout<<ans[i]<<" ";
```

CF1407D Discrete Centrifugal Jumps

- n 栋楼排成一行，第 i 栋楼高度为 h_i
- 楼 i 可以跳到楼 j ($i < j$)，当且仅当满足以下三个条件之一：
 - $i + 1 = j$
 - $\min(h_{i+1}, h_{i+2} \dots h_{j-1}) > \max(h_i, h_j)$
 - $\max(h_{i+1}, h_{i+2} \dots h_{j-1}) < \min(h_i, h_j)$
- 求从 1 号楼到 n 号楼最小跳跃次数
- $n \leq 3 \times 10^5, h_i \leq 10^9$

CF1407D Discrete Centrifugal Jumps

- 显然有简单的动态规划， $dp[i]$ 表示到 i 号楼最小跳跃次数
- $dp[j] = \min (dp[i]+1)$ 其中 i 能跳到 j
- 怎么看哪些 i 能跳到 j ？

CF1407D Discrete Centrifugal Jumps

- 怎么看哪些 i 能跳到 j ?
- $i = j - 1$ 肯定可以
- 条件 2 : $\min(h_{i+1}, h_{i+2} \dots h_{j-1}) > \max(h_i, h_j)$
- 遇到这种两个数的 $\max$ 分情况讨论
- 情况 1: h_i 更大, 那么 $i+1$ 到 $j-1$ 所有数都比 h_i 大, 换言之 j 是 i 右边第一个小于等于自己的数
- 情况 2: h_j 更大, 那么 $i+1$ 到 $j-1$ 所有数都比 h_j 大, 换言之 i 是 j 左边第一个小于等于自己的数
- 这两种情况显然都可以直接用单调栈维护, 条件 3 同理

CF1407D Discrete Centrifugal Jumps

- 参考转移

```
for(int i=1;i<=n;i++)dp[i]=n;
dp[1]=0;
for(int i=1;i<=n;i++){
    dp[i]=min(dp[i],dp[i-1]+1);//条件1
    if(lst[i][0])dp[i]=min(dp[i],dp[lst[i][0]]+1);//条件2情况1
    if(lst[i][1])dp[i]=min(dp[i],dp[lst[i][1]]+1);//条件3情况1
    if(nxt[i][0]<=n)dp[nxt[i][0]]=min(dp[nxt[i][0]],dp[i]+1);//条件2情况2
    if(nxt[i][1]<=n)dp[nxt[i][1]]=min(dp[nxt[i][1]],dp[i]+1);//条件3情况2
}
```

滑动窗口问题

- 假定有一个长度为 k 不断在序列上移动的窗口，我们如何实时维护该窗口内的最大值和最小值？
- 这两个问题是类似的，我们还是只考虑最大值
- 核心观察：如果我们从左到右维护这个过程，那假如一个数在另外一个数左边，还比这个数小，那这个数之后就永远没有用了（**单调性**）

滑动窗口问题

- 这个问题的单调性和前面的区别在于，虽然仍然是后进先弹出，但是先进的如果离右端点长度已经超过了 k ，也需要弹出，也就是两端需要弹出，因此使用单调（双端）队列
- 从左到右处理，假设当前窗口的右端点滑到了 i ：
- 如果队首的元素已经离右端点超过了 k ，弹出队首
- 按照规则，把队尾所有小于等于当前元素的都弹出
- 同样的，此时队列中的元素都是单调的，取队首即可

```
for(int i=1;i<=n;i++){
    while(q[hd]+k<=i)hd++;
    while(hd<=tl&& a[q[tl]]<=a[i])tl--;
    q[++tl]=i;
}
```

[ABC352D] Permutation Subsequence

给出长度为 n 的排列 p 和正整数 k 。

若一个长为 k 的子序列 $p_{i_1}, p_{i_2} \dots p_{i_k}$ 满足他们可以在重排之后成为连续的 k 个正整数，则称这个子序列是好的。

求出所有好的子序列中 $i_k - i_1$ 的最小值，可以证明答案一定存在。

$$1 \leq k \leq n \leq 2 \times 10^5$$

[ABC352D] Permutation Subsequence

- 依旧需要对问题的视角进行转化，我们枚举最终的答案是从 v 开始到 $v + k - 1$ 组成的连续正整数
- 求出 q_i 表示 i 在 p 中出现的位置，那么 $i_k - i_1$ 的值为：
$$\max_{i=v}^{v+k-1} (q_i) - \min_{i=v}^{v+k-1} (q_i)$$
- 这是非常经典的滑动窗口问题，使用单调队列维护即可

CF1195E OpenStreetMap

给出一个 $n \times m$ 的矩阵，求出其所有大小为 $a \times b$ 的子矩阵的最小值之和。

$1 \leq n, m \leq 3000$

CF1195E OpenStreetMap

- 对于高维的问题需要考虑如何降维为序列上的问题
- 求出 $c_{i,j} = \min_{k=j-b+1}^j w_{i,k}$ （只考虑每一行）
- 再求出 $d_{i,j} = \min_{k=i-a+1}^i c_{k,j}$ （只考虑每一列）
- 此时得到的 $d_{i,j}$ 就是以 (i,j) 为右下角的矩阵的最小值
- 只需要先对每一行做一次单调队列，再对每一列做一次单调队列即可。

CF1195E OpenStreetMap

- 参考代码

```
for(int i=1;i<=n;i++){//对每一行做单调队列
    hd=1;tl=0;
    w[i][0]=inf;
    for(int j=1;j<=m;j++){
        while(hd<=tl&&q[hd]+b<=j)hd++;
        while(hd<=tl&&w[i][q[tl]]>w[i][j])tl--;
        q[++tl]=j;
        c[i][j]=w[i][q[hd]];
    }
}

for(int j=1;j<=m;j++){//对每一列做单调队列
    hd=1;tl=0;
    for(int i=1;i<=n;i++){
        while(hd<=tl&&q[hd]+a<=i)hd++;
        while(hd<=tl&&c[q[tl]][j]>c[i][j])tl--;
        q[++tl]=i;
        d[i][j]=c[q[hd]][j];
    }
}
```

P1080 [NOIP 2012 提高组] 国王游戏

给出 $n + 1$ 组数对 (a_i, b_i) ，现要将他们排成一定的顺序 (a'_i, b'_i) ，要求：

- (a_1, b_1) 位置不变
- 设 c_i 为 $a'_1 \cdots a'_{i-1}$ 的积，最小化 $\lfloor \frac{c_i}{b'_i} \rfloor$ 的最大值

$$1 \leq n \leq 1000, 1 \leq a_i, b_i < 10000$$

P1080 [NOIP 2012 提高组] 国王游戏

- 很明显是一道贪心题，我们需要按某种顺序排序所有的 (a_i, b_i)
- 但是由于有 a, b 两个因素，排序的策略并不好直接观察
- 这个时候就可以使用邻项交换法

P1080 [NOIP 2012 提高组] 国王游戏

- 我们假设已经按照某种顺序排好了 (a_i, b_i) ，如果这种顺序是最优的，那么交换任意相邻两项结果一定不会更好
- 例如要交换 i 和 $i+1$ ，那么应当有：
$$\max\left(\frac{c_i}{b_i}, \frac{c_i a_i}{b_{i+1}}\right) \leq \max\left(\frac{c_i}{b_{i+1}}, \frac{c_i a_{i+1}}{b_i}\right)$$
- 经过化简会发现，这等价于 $a_i b_i \leq a_{i+1} b_{i+1}$ 时，交换不会变得更好
- 那么此时我们就直接推导出了这道题目的排序方式，按此方式排序后计算答案即可
- 注意本题需要使用高精度

P2209 [USACO13OPEN] Fuel Economy S

- 一辆油箱容量为 G 的卡车要从数轴上从0走到 D ，走一单位消耗1单位油
- 初始有 B 单位油，
- 路上有 n 个加油站，第 i 个加油站位于 x_i ，单位油价格为 y_i ，
- 问从起点到终点的最小花费
- $n \leq 5 \times 10^4, D \leq 10^9, G \leq 10^6$

P2209 [USACO13OPEN] Fuel Economy S

- 对于这种决策比较复杂的问题，优先考虑最优决策是什么样子的
- 问题的关键是在哪些加油站加油，以及加多少油
- 考虑在加油站 i :
- 假定后面第一个比其便宜的加油站是 j ，那么我们显然尽量在 j 去加油，因此只需要在 i 加油到足够到 j 就可以了
- 如果即使加满了也到不了 j 呢？
- 那说明路上还得再加油，但是从 i 到 j 已经没有更便宜的加油站了，因此在 i 加满是最优的选择
- 模拟即可

P2209 [USACO13OPEN] Fuel Economy S

- 参考代码

```
top=0;
S[0]=n+1;
for(int i=n;i>=1;i--){
    while(top&&S[top].y>=a[i].y)top--;
    nxt[i]=S[top];
    S[++top]=i;
}
```

```
int now=B;
a[n+1].x=D;
ll ans=0;
for(int i=1;i<=n+1;i++){
    now=a[i].x-a[i-1].x;
    if(now<0){
        cout<<-1<<endl;
        return 0;
    }
    int need=a[nxt[i]].x-a[i].x;
    if(need<=now)continue;
    if(need>G){
        ans+=1ll*(G-now)*a[i].y;
        now=G;
    }
    else{
        ans+=1ll*(need-now)*a[i].y;
        now=need;
    }
}
```

P2209 [USACO13OPEN] Fuel Economy S

- 这种题目的另一种思路是带有反悔的贪心
- 我们考虑这样一个过程：加油本身不花钱，耗油的时候花费对应购买时的单价
- 那么首先每次耗油的时候，我们肯定耗的是价格最低的那种油
- 如果到达一个加油站，那么此时还没用掉的油如果单价高于这里的价格，我们就可以用当前价格的油替换原来的（反悔）
- 同样的，模拟这个过程即可

P2209 [USACO13OPEN] Fuel Economy S

- 参考代码

```
q[hd=tl=1]=make_pair(B,0);
int now=B;
a[n+1].x=D;
for(int i=1;i<=n+1;i++){
    int need=a[i].x-a[i-1].x;
    if(now<need){
        cout<<-1<<endl;
        return 0;
    }
    while(hd<=tl&&need){
        ans+=1ll*min(need,q[hd].first)*q[hd].second;
        if(need<=q[hd].first){
            q[hd].first-=need;
            now-=need;
            need=0;
        }
        else{
            need-=q[hd].first;
            now-=q[hd].first;
            hd++;
        }
    }
    while(hd<=tl&&q[tl].second>=a[i].y){
        now-=q[tl].first;
        tl--;
    }
    q[++tl]=make_pair(G-now,a[i].y);
    now=G;
}
cout<<ans<<endl;
```

P1484 种树

- 给定一个序列，要求从中选至多 k 个数且不存在两个相邻的被选，问选到的和的最大值
- $n, k \leq 3 \times 10^5$

P1484 种树

- 常规的贪心比较难做，依旧考虑反悔贪心
- 考虑 $.x.x.x.x.$ 这样的形式， x 表示目前选了
- 假如要反悔某个 x ，那就是 $.x.x.x.x. \rightarrow x.x.x.x.x$ 正好多选一个
- 那么我们维护一个堆，初始放入所有 a_i ，执行至多 k 次每次取最大的一个，然后把这个左边+这个右边-这个本身放入堆中，即可实现反悔

P1484 种树

- 参考实现

```
while(m--){
    while(vis[q.top().x])q.pop();
    node t=q.top();
    q.pop();
    vis[nxt[t.x]]=vis[lst[t.x]]=1;
    now+=t.val;
    t.val=-t.val+val[nxt[t.x]]+val[lst[t.x]];
    val[t.x]=t.val;
    lst[nxt[nxt[t.x]]]=t.x;
    nxt[t.x]=nxt[nxt[t.x]];
    nxt[lst[lst[t.x]]]=t.x;
    lst[t.x]=lst[lst[t.x]];
    q.push(t);
    ans=max(now,ans);
}
```

倍增

- 最近公共祖先
- 一个点的某一级祖先是确定的，预处理出每个点的第 2^k 级祖先是哪个点。
- 此时已经可以求任意点的任意一级祖先
- 本质是把一个数字拆分为若干个二进制次幂
- 我们还可以求两个点的最近公共祖先
- 这是用倍增求解形如“跳0~k-1步不满足条件，k及以上步满足条件，找k”的问题

倍增

- st 表
- 预处理出每个左端点区间长度为 2^k 的信息。
- 由于 max,min 等信息可以合并，我们查询时取 $t = \log_2(len)$ 分别从左端点和右端点查询区间长度为 2^t 的区间，由于 $2^{t+1} > len$ ，一定能覆盖整个区间
- 如果这个信息不可算重（比如求和），可以类似倍增求 k 级祖先做到单次 $O(\log)$ 查询

CF1142B Lynyrd Skynyrd

给出一个长度为 n 的排列 p 和一个长度为 m 的数组 a 。

q 次询问，每次给出一个区间 $[l, r]$ ，问 $a[l \cdots r]$ 是否存在一个子序列是 p 的一个循环移位。

$1 \leq n, m, q \leq 2 \times 10^5$

CF1142B Lynyrd Skynyrd

- 不论怎么循环移位，一个数字后面该接哪个数字是确定的， p_i 后面一定是 $p_{i \bmod n + 1}$
- 匹配的原则一定是文本串找最近的位置匹配模式串下一位
- 因此，我们知道 a_i 后面要接哪个数字，也知道要匹配最近的一位，因此可以知道如果 a_i 参与匹配，下一个参与匹配的是谁（记为 nxt_i ）
- 匹配的过程：从一个位置 i 开始，跳 $n - 1$ 次 nxt_i 到达 i' ，那么 $[i, i']$ 以及所有包含这个区间的区间都可以找到一个序列是 p 的循环移位

CF1142B Lynyrd Skynyrd

- 换言之，我们只需要预处理设 i 跳 $n - 1$ 步后到达的位置为 b_i ，那么显然只需要满足 $\min_{i=l}^r b_i \leq r$ 即可（这可以使用 ST 表简单处理）
- 现在已知每个位置跳的下一步是谁，要求某个位置跳任意步是谁，直接套用倍增求 LCA 的模式即可

CF1142B Lynyrd Skynyrd

- 代码实现（观察到 $b_i \geq i$ ，因此不需要 ST 表，求后缀 min 即可）

```
for(int i=1;i<=n;i++)match[p[i]]=p[i%n+1];
for(int i=1;i<=n;i++)lst[i]=m+1;
for(int i=m;i>=1;i--){
    nxt[i][0]=lst[match[a[i]]];
    lst[a[i]]=i;
}
for(int j=1;(1<<j)<=m;j++){
    for(int i=1;i+(1<<j)-1<=m;i++){
        nxt[i][j]=nxt[nxt[i][j-1]][j-1];
        if(!nxt[i][j])nxt[i][j]=m+1;
    }
}
for(int i=1;i<=m;i++){
    int t=n-1,now=i;
    for(int j=19;j>=0;j--){
        if((t&(1<<j)))now=nxt[now][j];
    }
    if(!now)now=m+1;
    b[i]=now;
}
for(int i=m-1;i>=1;i--)b[i]=min(b[i],b[i+1]);
while(q--){
    int l,r;
    cin>>l>>r;
    if(b[l]<=r)cout<<1;
    else cout<<0;
}
```

CF1848F Vika and Wiki

给出序列 a ，定义一次变换为（所有位置同时更新）：

- $a_i \leftarrow a_i \oplus a_{i+1} (0 \leq i < n - 1)$
- $a_{n-1} \leftarrow a_{n-1} \oplus a_0$

求出最少经过几次变换可以将 a_i 全部变成 0，或输出 -1 表示不可能全部变成 0。

保证 $n = 2^k (0 \leq k \leq 20), 0 \leq a_i \leq 10^9$

CF1848F Vika and Wiki

- 先通过手玩找一下规律：

0	1	2	3	4	5	6	7
$0 \wedge 1$	$1 \wedge 2$	$2 \wedge 3$	$3 \wedge 4$	$4 \wedge 5$	$5 \wedge 6$	$6 \wedge 7$	$7 \wedge 0$
$0 \wedge 2$	$1 \wedge 3$	$2 \wedge 4$	$3 \wedge 5$	$4 \wedge 6$	$5 \wedge 7$	$6 \wedge 0$	$7 \wedge 1$
$0 \wedge 1 \wedge 2 \wedge 3$	$1 \wedge 2 \wedge 3 \wedge 4$	$2 \wedge 3 \wedge 4 \wedge 5$	$3 \wedge 4 \wedge 5 \wedge 6$	$4 \wedge 5 \wedge 6 \wedge 7$	$5 \wedge 6 \wedge 7 \wedge 0$	$6 \wedge 7 \wedge 0 \wedge 1$	$7 \wedge 0 \wedge 1 \wedge 2$
$0 \wedge 4$	$1 \wedge 5$	$2 \wedge 6$	$3 \wedge 7$	$4 \wedge 0$	$5 \wedge 1$	$6 \wedge 2$	$7 \wedge 3$
...							

- 经过 2^d 次操作后, a_i 会变为: $a_i \oplus a_{(i+2^d) \bmod n}$
- $d = k$ 所有 a_i 一定会变成 0, 故一定有解

CF1848F Vika and Wiki

- 那怎么确定最早全变为 0 的时间呢?
- 一旦全变为 0，之后无论怎么操作都一直是 0
- 我们可以知道序列经过二的幂次次操作后的结果
- 操作可以拆分
- 这完全符合倍增求解形如“跳 $0 \sim k-1$ 步不满足条件， k 及以上步满足条件，找 k ”的问题，使用倍增即可

CF1848F Vika and Wiki

- 参考代码

```
int ans=0;
for(int j=k-1;j>=0;j--){
    for(int i=0;i<n;i++)b[i]=a[i]^a[(i+(1<<j))%n];
    bool flag=1;
    for(int i=0;i<n;i++)flag&=(b[i]==0);
    if(!flag){
        for(int i=0;i<n;i++)a[i]=b[i];
        ans+=(1<<j);
    }
}
cout<<ans+1<<endl;
```

P15352 [COCI 2025/2026 #4] 魔术 / Magija

有一个操作池，初始为空。有 Q 个事件：

- 1 x : 对排列 $[1, 2, \dots, N]$
，进行任意次（可以不进行）操作池中的操作，
求出操作完后 x 最终下标的最小值和最大值。
- 2 $[l, r, \text{len}]$: 向操作池中加入一个操作 (l, r, len) 含义为对于 $i = 0, \dots, \text{len} - 1$ ，交换 p_{l+i} 与 p_{r+i} （保证两个交换区间不相交）

$n, q \leq 2 \times 10^5$ 32MB

P15352 [COCI 2025/2026 #4] 魔术 / Magija

- 当 x 和 y 可以交换，由于可以任意使用，它们的值可以任意换
- 把这种等价关系看成边，那么某一个元素可达的是其所在的整个连通块
- 直接使用并查集维护，复杂度 $O(nq)$

P15352 [COCI 2025/2026 #4] 魔术 / Magija

- 如何优化？引入二进制拆分的思想
- 我们要做的事情是，对于 l 开始一段和 r 开始一段一对一的连边
- 任意长度是不好做的，但如果只有二的幂次呢？

P15352 [COCI 2025/2026 #4] 魔术 / Magija

- 我们用 $[i, j]$ 表示 i 开头长度为 2^j 的区间
- 当 $[l, r, \text{len}]$ 的操作出现时，把 len 拆分为若干个二进制次幂依次连边
- 问题是这种关系如何从大区间下放给小区间
- 我们考虑当前是 $\text{merge}(l, r, 2^k)$
- 如果 $[l, k]$ 和 $[r, k]$ 已经连过了，直接返回
- 否则递归 $\text{merge}(l, r, 2^{k-1})$ 和 $\text{merge}(l + 2^{k-1}, r + 2^{k-1}, 2^{k-1})$

P15352 [COCI 2025/2026 #4] 魔术 / Magija

- 考虑这样做的复杂度?
- 因为已经连过的我们会直接返回, 因此只需要考虑有效的连边次数
- 考虑每一层, 有 n 个点, 有效连边次数 $O(n)$, 一共 $\log(n)$ 层

P15352 [COCI 2025/2026 #4] 魔术 / Magija

- 参考实现

```
void merge(int l,int r,int k){
    if(k<0)return ;
    if(gf(l,k)==gf(r,k))return ;
    if(k>0){
        fa[gf(l,k)][k]=gf(r,k);
    }
    else{
        g[gf(r,k)].mn=min(g[gf(l,k)].mn,g[gf(r,k)].mn);
        g[gf(r,k)].mx=max(g[gf(l,k)].mx,g[gf(r,k)].mx);
        fa[gf(l,k)][k]=gf(r,k);
    }
    merge(l,r,k-1);
    merge(l+(1<<(k-1)),r+(1<<(k-1)),k-1);
}

for(int i=1;i<=q;i++){
    int op,x,l,r,len;
    cin>>op;
    if(op==1){
        cin>>x;
        cout<<g[gf(x,0)].mn<<" "<<g[gf(x,0)].mx<<"\n";
    }else{
        cin>>l>>r>>len;
        for(int j=17;j>=0;j--){
            if(len>>j&1)
                merge(l,r,j),l+=(1<<j),r+=(1<<j);
        }
    }
}
```

序列分治

- 一种比较经典的思想，可以用在很多数据结构上
- 如果要解决 $[L,R]$ 的问题，划分一个中点，先解决 $[L,mid]$ 和 $[mid+1,R]$ 的问题
- 如果 $[L,mid]$ 对 $[mid+1,R]$ 有影响，在这一层我们再额外处理两个子区间之间的影响
- 如果 $mid = (L+R)/2$ ，这就是经典的线段树结构

CF817D Imbalanced Array

- 给出长度为 n 的序列 a , 求:
- $\sum_{l=1}^n \sum_{r=l}^n (\max_{i=l}^r a_i - \min_{i=l}^r a_i)$
- $n \leq 10^6$

CF817D Imbalanced Array

- 两个式子可以拆开计算
- $\sum_{l=1}^n \sum_{r=l}^n \max_{i=l}^r a_i - \sum_{l=1}^n \sum_{r=l}^n \min_{i=l}^r a_i$
- 这两个式子明显是类似的，只用考虑其中一个怎么算

CF817D Imbalanced Array

- 例如计算 $\max$
- 考虑现在计算 $[l, r] \in [L, R]$ 的所有答案
- 先计算 $[l, r] \in [L, \text{mid}]$ 和 $[l, r] \in [\text{mid} + 1, R]$
- 那么现在只需要计算 $l \in [L, \text{mid}], r \in [\text{mid} + 1, R]$ 的情况
- 此时 $[l, r]$ 的最大值可以分为 $[l, \text{mid}]$ 的最大值和 $[\text{mid} + 1, r]$ 的最大值

CF817D Imbalanced Array

- 此时 $[l, r]$ 的最大值可以分为 $[l, \text{mid}]$ 的最大值和 $[\text{mid} + 1, r]$ 的最大值
- 记录 suf_l 和 pre_r 分别表示上面两者，那么我们要计算的其实是：

$$\sum_{l=L}^{\text{mid}} \sum_{r=\text{mid}+1}^R \max_{i=l}^r a_i = \sum_{l=L}^{\text{mid}} \sum_{r=\text{mid}+1}^R \max(\text{suf}_l, \text{pre}_r)$$

- 那么枚举 l ，二分/双指针 $\text{suf}_l \leq \text{pre}_r$ 的位置，统计答案即可。
- 参考线段树，我们知道递归的次数是 $n \log n$ ，总复杂度 $O(n \log^2 n) / O(n \log n)$

CF817D Imbalanced Array

- 参考代码:

```
11 solve_max(int L,int R){
    if(L==R)return a[L];
    int mid=(L+R)/2;
    11 res=solve_max(L,mid)+solve_max(mid+1,R);
    suf[mid]=a[mid];for(int i=mid-1;i>=L;i--)suf[i]=max(suf[i+1],a[i]);
    pre[mid+1]=a[mid+1];for(int i=mid+2;i<=R;i++)pre[i]=max(pre[i-1],a[i]);
    int j=mid+1;
    sum_pre[R+1]=0;
    for(int i=R;i>=mid;i--)sum_pre[i]=sum_pre[i+1]+pre[i];
    for(int i=mid;i>=L;i--){
        while(j<=R&&pre[j]<=suf[i])j++;
        res+=1ll*suf[i]*(j-mid-1);
        res+=sum_pre[j];
    }
    return res;
}
```

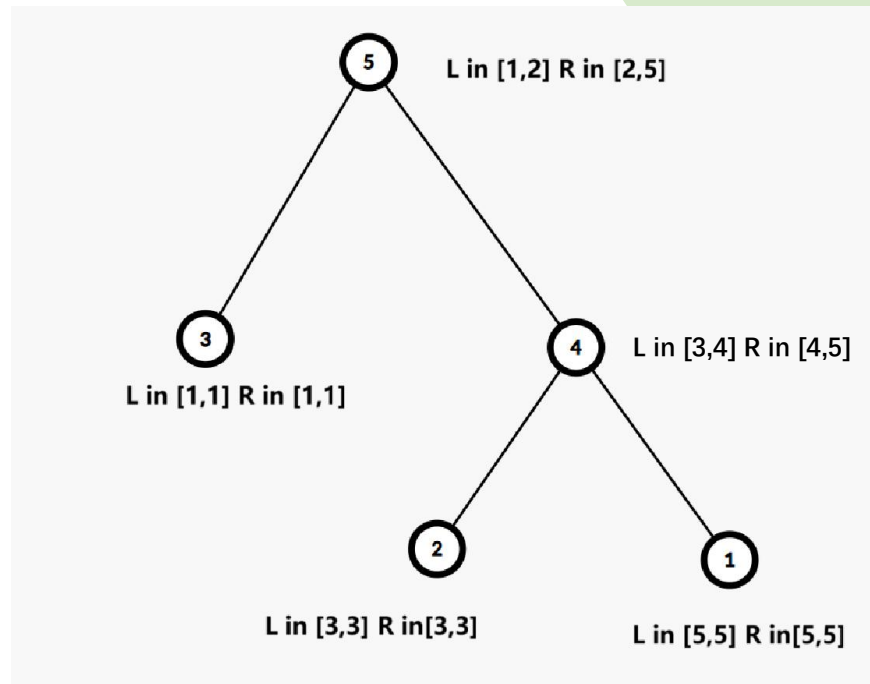
CF817D Imbalanced Array

- 这个题也可以用我们之前讲的单调栈完成
- 例如求解 $\max$, 我们可以用单调栈求每个数 a_i 最为最大值的最大区间 $[L_i, R_i]$

$$\sum_{l=1}^n \sum_{r=l}^n \max_{i=l}^r a_i = \sum_{i=1}^n a_i (i - L_i) (R_i - i)$$

CF817D Imbalanced Array

- 实际上，如果我们把这个过程画出来，会发现也是一个树形的分治结构，其实就是笛卡尔树



e.g : 序列 [3,5,2,4,1] 对应的笛卡尔树

CF873D Merge Sort

- 考虑这样一个归并排序的过程：
- MergeSort(l,r)时，如果 $a[l \cdots r]$ 本身有序，直接返回
- 否则递归 MergeSort(l,mid) 和 MergeSort(mid,r)，随后合并两边的结果
- 初始调用 MergeSort(0,n)
- 输入 n,k ，要求构造一个排列 a ，使 MergeSort 函数恰好被调用 k 次，无解输出 -1

CF873D Merge Sort

- 初始要递归一次 $(1, n)$ ，减掉之后每次调用都会产生两个分支，因此 $k-1$ 一定是偶数
- 考虑利用分治的思想递归构造
- 如果当前要构造 (L, R) ，这个区间内需要调用 M 次（ M 为偶数）
- $M = 0$ 时， L 到 R 一定是递增的
- $M > 0$ 时，递归构造 (L, mid) $(\text{mid}+1, R)$ ，调用次数分别为两个接近的偶数即可（具体见实现）

CF873D Merge Sort

- 参考代码（注意题目是 0-index, 左闭右开）

```
void solve(int L,int R,int M){
    if(!M){
        for(int i=L;i<R;i++)a[i]=++tot;
        return ;
    }
    if(L+1==R){
        cout<<-1<<endl;
        exit(0);
    }
    int mid=(L+R)/2;
    int a=(M-2)/2,b=(M-2)/2;
    if(a%2==1){
        b++;
        a--;
    }
    solve(mid,R,b);
    solve(L,mid,a);
}
```